

SCHOOL OF TECHNOLOGY

WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

PROJECT TITLE

DESIGN AND IMPLEMENTATION OF A DATABASE
MANAGEMENT SYSTEM FOR HOSPITAL APPOINTMENT
AND PATIENT CARE MANAGEMENT SYSTEM

Program	B.Tech (Computer Science & Engineering) — 2nd Year
Section	AIML Rhinos
Faculty	Professor Sahithi
Date of Submission	October 05, 2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102098	Himanshu Pandit	+91 98765 43210	himanshu.p@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, **Professor Sahithi**, for the valuable guidance, insightful suggestions, and continuous support throughout the conceptual design, normalization, schema engineering, and software application development phases of this project. The guidance helped me understand the practical application of core database concepts such as classical Entity-Relationship (ER) modelling, relational schema decomposition up to Third Normal Form (3NF), declarative and procedural integrity constraints, SQL query formulation, and end-to-end database-backed application engineering.

This PBL activity has significantly strengthened my theoretical comprehension of relational database concepts and provided invaluable experiential learning in architecting, validating, and deploying a real-world enterprise database application.

Himanshu Pandit

Roll Number: 25WU0102098

B.Tech CSE — 2nd Year (Section: AIML Rhinos)

Woxsen University

UNDERTAKING

I hereby declare that the project entitled “**Design and Implementation of a Database Management System for Hospital Appointment and Patient Care Management System**”, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty, **Professor Sahithi**.

I undertake that the project report, relational database design, SQL scripts, application source code, execution results, and supporting analytical materials submitted by me have been prepared for academic purposes. Wherever information, design patterns, datasets, or academic resources have been referred to, appropriate acknowledgement and citations have been provided in accordance with IEEE formatting standards.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have adhered to all academic integrity and project guidelines prescribed by Woxsen University.

Date: October 05, 2026

Place: Hyderabad, India

Signature: _____

Name: Himanshu Pandit

Roll Number: 25WU0102098

Program: B.Tech CSE 2nd Year (AIML Rhinos)

TABLE OF CONTENTS

Contents	Page No.
Abstract	9
LIST OF TABLES	6
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	10
1.1 Problem Statement	10
1.2 Objectives	10
1.3 Scope and Assumptions	11
1.4 Proposed Solution Strategy	12
1.5 Functional Requirements	12
1.6 Entity-Relationship Diagram (ERD)	14
1.7 Entity and Relationship Summary	14
2. LOGICAL DATABASE DESIGN	16
2.1 Relational Schema	16
2.2 Keys and Integrity Constraints	17
2.3 Functional Dependencies	19
2.4 Normalization up to Third Normal Form (3NF)	20
2.5 Data Dictionary	22
2.6 Implementation of Logical Design: DDL and Sample Data	25
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	27
3.1 Application Overview	27
3.2 Development Environment and Database Connectivity	27
3.3 Application Modules and Database Mapping	28
3.4 CRUD Operations	30
3.5 Business Rules, Validation and Error Handling	30
3.6 Reports and Database Operations	31
3.7 Application Screens	32
3.8 Testing and Results	36
4. CONCLUSION	38
4.1 Limitations and Future Enhancements	38
5. REFERENCES	40
6. APPENDICES	41

Appendix A - Complete Database Script	41
Appendix B - Application Source Code	42
Appendix C - Additional Test Data and Outputs	42

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	11
Table 2	Proposed Solution Strategy	12
Table 3	Functional Requirements	12
Table 4	Entity and Relationship Summary	14
Table 5	Relational Schema	16
Table 6	Keys and Integrity Constraints	17
Table 7	Normalization Status up to 3NF	20
Table 8	Data Dictionary	22
Table 9	Database Implementation Environment	25
Table 10	Application Development Environment	27
Table 11	Application Module-to-Database Mapping	28
Table 12	CRUD Operations	30
Table 13	Business Rules, Validation and Error Handling	30
Table 14	Reports and Database Operations	31
Table 15	Test Cases	36

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	14
Figure 2	CarePulse Patient Management & Clinical Overview Dashboard	32
Figure 3	Doctor Appointment Scheduling & Collision Prevention Interface	33
Figure 4	Inpatient Ward Census & Active Bed Allocation Map	34
Figure 5	Financial Billing Settlement & Revenue Transaction Module	35

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BCNF	Boyce-Codd Normal Form
CLI	Command-Line Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER / ERD	Entity-Relationship / Entity-Relationship Diagram
FD	Functional Dependency
FK	Foreign Key
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ICD-10	International Classification of Diseases, 10th Revision
ICU	Intensive Care Unit
JSON	JavaScript Object Notation
OPD	Outpatient Department
PBL	Project-Based Learning
PK	Primary Key
RDBMS	Relational Database Management System
REST	Representational State Transfer

Acronym	Expansion
SQL	Structured Query Language
UI / UX	User Interface / User Experience
URI / URL	Uniform Resource Identifier / Uniform Resource Locator

ABSTRACT

Modern hospitals manage thousands of concurrent transactions across fragmented clinical and administrative departments, resulting in doctor appointment collisions, bed double-booking, dispersed medical histories, and revenue leakage. The primary objective of this project is to design, implement, and validate a centralized, production-grade Relational Database Management System (RDBMS) paired with an interactive Python application for hospital appointment and patient care management. The conceptual architecture is formalized using Classical Chen ER modeling across 14 relational tables, fully normalized up to Third Normal Form (3NF) to eliminate partial and transitive functional dependencies. The SQLite relational engine enforces database triggers that mathematically prevent overlapping appointments, enforce a strict one-active-patient-per-bed constraint, validate chronological admission dates, and cap payment settlements. A zero-dependency Python web application controller provides live patient searching, appointment scheduling, consultation entry, bed census management, payment recording, and five pre-compiled analytical views. Comprehensive verification demonstrates flawless data integrity, transactional consistency, and intuitive user workflows. The completed application provides an enterprise-ready, scalable prototype establishing clinical coherence and operational transparency in healthcare informatics.

KEYWORDS: Relational Database, Entity-Relationship Diagram, 3NF Normalization, Integrity Constraints, Python Web Application, Hospital Information System.

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

This chapter presents the operational challenges, project objectives, system boundaries, architectural strategy, functional requirements, and conceptual Entity-Relationship design developed during Review 1 of the Hospital DBMS project.

1.1 Problem Statement

Modern hospital operations encompass highly interdependent clinical and administrative workflows, including outpatient registration, physician duty roster configuration, specialist appointments, physical examination recording, prescription and diagnostic test ordering, inpatient ward admissions, bed tracking, and financial invoice settlement. In legacy paper-based clinics or decentralized departmental electronic spreadsheets, records are siloed within discrete departments. This architectural fragmentation introduces severe operational anomalies: specialist physicians are regularly double-booked for concurrent time slots; inpatient ward allocations suffer from double-booking conflicts where multiple patients are concurrently assigned to the same physical hospital bed; clinical consultations, diagnoses, and lab results are dispersed across unlinked records; and billing cashiers face significant revenue leakage due to unbilled diagnostic orders and uncoordinated patient discharge clearances.

Consequently, hospital administrators lack consolidated visibility into patient medical trajectories, daily bed occupancy rates, doctor workloads, and pending revenue. To eliminate these critical deficits, there is an urgent imperative for a centralized, fully normalized Relational Database Management System (RDBMS) governed by schema-level integrity constraints, automated triggers, and an intuitive software application interface that enforces enterprise business rules at the persistence tier.

1.2 Objectives

The primary objectives of this Project-Based Learning development are:

1. **Design** a rigorous conceptual database model using Classical Chen Notation capturing 14 clinical and administrative entities and their cardinalities.
2. **Decompose** the database schema up to Third Normal Form (3NF) to guarantee lossless join decomposition and complete preservation of functional dependencies.
3. **Implement** automated database triggers and declarative check constraints to prevent doctor schedule overlaps, enforce single active occupancy per hospital bed, and validate financial settlements.

- 4. **Develop** an interactive, zero-dependency Python application providing a responsive web dashboard and terminal interface for complete clinical CRUD workflows.
- 5. **Validate** transactional consistency and query execution performance across outpatient, inpatient, laboratory, and billing domains through comprehensive positive and negative test cases.

1.3 Scope and Assumptions

The operational boundaries, system scope, exclusions, and environmental assumptions are summarized in Table 1.

Table 1. Scope and Assumptions

Item	Description
In Scope	• Centralized outpatient patient registration, demographic updates, and search. • Departmental structure and doctor duty roster configuration. • Outpatient appointment booking with automated time-window collision checks. • Clinical consultation recording, ICD-10 diagnostic coding, and prescription items. • Diagnostic laboratory test catalog management and investigation tracking. • Inpatient ward census, bed assignment, and single-occupancy bed locking. • Comprehensive invoice generation, partial/full payment recording, and receipts. • Five compiled SQL analytical reporting views.
Out of Scope / Exclusions	• Real-time medical telemetry equipment integration (e.g., ICU monitor feeds). • Complex pharmaceutical warehouse stock replenishment supply chain logistics. • External insurance company claim EDI clearinghouse integration. • Biometric staff attendance hardware integration.
Assumptions	• Doctors operate on discrete, pre-configured shift schedules with designated consultation slot intervals. • Each physical bed belongs to exactly one ward and accommodates at most one active inpatient at any point in time. • Inpatient discharge must be chronologically greater than or equal to admission timestamp. • Monetary currencies are handled in standard decimal currency format (INR ₹) with non-negative bounds.

1.4 Proposed Solution Strategy

The proposed solution implements a Three-Tier Software Architecture comprising an HTML5/CSS3 Single Page Application presentation layer, a lightweight Python HTTP controller business logic layer, and an ACID-compliant SQLite relational database tier. Table 2 details the architectural elements, user roles, core workflows, and business rules.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Hospital Administrator, Specialist Doctor, Patient / Attendant, Staff Nurse, Lab Technician, and Billing Cashier.
Major Application Modules	Patient Registry Module, Doctor Scheduling Module, Appointment Management Module, Clinical Consultation & Prescription Module, Ward Bed Census Module, Billing & Payment Settlement Module, and Analytical Reporting Module.
Proposed Workflow	Patient Demographic Intake → Specialist Selection & Slot Booking → Clinical Consultation & Diagnosis → Diagnostic/Prescription Ordering → Inpatient Ward Bed Allocation → Invoice Generation & Payment Settlement.
Important Business Rules	1. No doctor can have overlapping appointments within the same time window. 2. A physical hospital bed cannot host more than one active admitted patient simultaneously. 3. Discharge date must be chronologically greater than or equal to admission date. 4. Cumulative payments recorded against a bill cannot exceed net payable amount.
Expected Reports / Outputs	Patient Longitudinal Medical History, Doctor Rosters & Capacity, Pending Lab Investigation Queue, Ward Bed Occupancy Census, and Outstanding Accounts Receivable Ledger.

1.5 Functional Requirements

The system functional specifications are formally categorized into five primary requirements as tabulated in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Patient Demographic Management: Register unique patient profiles with validated contact numbers, date of birth, blood group, and emergency contact details. Enable dynamic multi-parameter searching.	High / Core Intake
FR-02	Roster & Appointment Scheduling: Maintain doctor weekly shift availability. Schedule appointments without slot collisions using database-level pre-insert validation.	High / Operational
FR-03	Clinical Consultation & Medication Ordering: Record clinical symptoms, physical examination findings, ICD-10 diagnostic classifications, and multi-item itemized prescriptions.	High / Clinical
FR-04	Inpatient Ward Bed Allocation: Visualize real-time operational bed status across ICU, General, and Private wards. Restrict admissions to vacant beds only with atomic state transition.	High / Inpatient

Req. ID	Functional Requirement	Remarks / Priority
FR-05	Financial Billing & Payment Settlement: Aggregate consultation, room, and diagnostic charges into itemized invoices. Record multi-channel payments (UPI, Card, Cash) with real-time balance tracking.	High / Financial

1.6 Entity-Relationship Diagram (ERD)

The conceptual model is formalized using Classical Chen Notation to clearly depict strong entities (rectangles), multi-valued and composite decomposed attributes (ovals), primary keys (underlined text), relationships (diamonds), and structural cardinalities (1:1, 1:M, M:N). Figure 1 displays the final, approved Entity-Relationship diagram representing all 14 clinical and administrative relations.

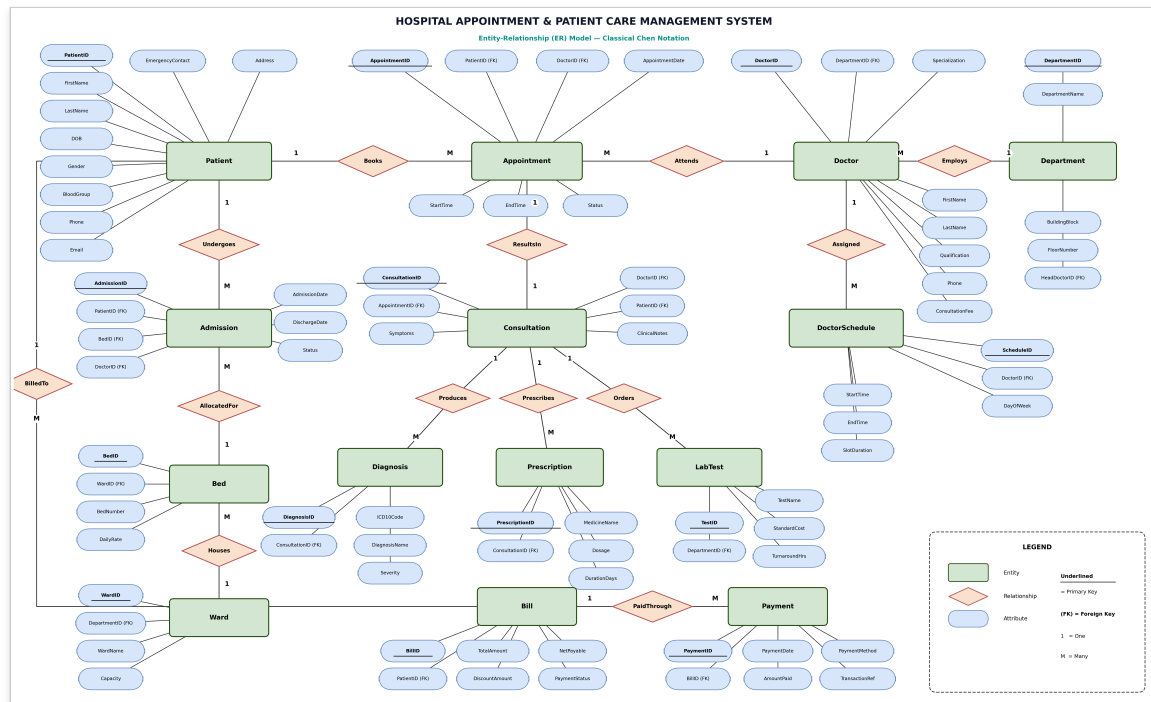


Figure 1. Entity-Relationship Diagram of the Proposed System

As illustrated in Figure 1, the conceptual design establishes clear separation of concerns across administrative, outpatient, clinical, diagnostic, and financial domains. Key structural constraints such as the strict 1:1 relationship between Appointment and Consultation, the 1:M containment of PrescriptionItem within Prescription, and the 1:M room occupancy link between Bed and Admission are preserved.

1.7 Entity and Relationship Summary

Table 4 provides an exhaustive description of every entity, its primary identifying key, key attributes, associated relationships, and structural cardinality.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Business Rule
Department	dept_id (PK), dept_name, building_block, floor_number, contact_ext, head_doctor_id.	Represents clinical and administrative divisions; 1 Department manages M Doctors.
Doctor	doctor_id (PK), dept_id (FK), first_name, last_name, specialization, fee, license_no.	Medical practitioners; 1 Doctor belongs to 1 Department; conducts M Appointments.
DoctorSchedule	schedule_id (PK), doctor_id (FK), day_of_week, start_time, end_time, slot_mins, max_patients.	Defines weekly working shifts and slot duration; 1 Doctor has M Schedules.
Patient	patient_id (PK), first_name, last_name, dob, gender, blood_group, phone, address.	Healthcare recipients; 1 Patient books M Appointments; undergoes M Admissions.
Appointment	appointment_id (PK), patient_id (FK), doctor_id (FK), date, start_time, end_time, status.	Outpatient booking; Enforces no overlapping slots for the same doctor.
Consultation	consultation_id (PK), appointment_id (FK), doctor_id (FK), patient_id (FK), symptoms, notes.	Clinical encounter; Exactly 1 Consultation originates from 1 Appointment.
Diagnosis	diagnosis_id (PK), consultation_id (FK), icd10_code, diagnosis_name, severity.	Clinical findings; 1 Consultation yields M Diagnoses (Primary & Secondary).
Prescription	prescription_id (PK), consultation_id (FK), prescription_date, advice.	Medical prescription header; 1 Consultation generates 1 Prescription.
PrescriptionItem	item_id (PK), prescription_id (FK), medicine_name, dosage, frequency, duration, quantity.	Decomposed atomic medication line items eliminating INF repeating groups.
LabTest	test_id (PK), dept_id (FK), test_name, test_code, standard_cost, turnaround_hours.	Catalog of laboratory diagnostic procedures offered by the hospital.
TestOrder	order_id (PK), consultation_id (FK), test_id (FK), order_date, status, result_value.	Diagnostic investigation orders dispatched from consultations.
Ward	ward_id (PK), dept_id (FK), ward_name, ward_type, floor_number, capacity.	Inpatient physical units (ICU, General, Private); 1 Ward contains M Beds.
Bed	bed_id (PK), ward_id (FK), bed_number, daily_rate, is_operational.	Physical hospital beds; Accommodates at most 1 active Admission at a time.
Admission	admission_id (PK), patient_id (FK), bed_id (FK), doctor_id (FK), admission_date, discharge_date.	Inpatient hospital stays; Enforces valid chronological stay durations.
Bill	bill_id (PK), patient_id (FK), consultation_id (FK), admission_id (FK), total, discount, tax, net.	Itemized financial statement; Enforces paid_amount ≤ net_payable.
Payment	payment_id (PK), bill_id (FK), amount_paid, payment_method, transaction_ref.	Financial remittances; Updates parent bill paid balance and payment status.

CHAPTER 2

LOGICAL DATABASE DESIGN

This chapter details the transformation of the conceptual Entity-Relationship model into a normalized relational schema, documents formal functional dependencies, presents formal 3NF normalization proofs, defines the comprehensive data dictionary, and highlights schema DDL implementation.

2.1 Relational Schema

The relational database schema consists of 14 base relations, 3 database triggers, and 5 compiled analytical views. Table 5 lists all relations derived from the ER model along with their operational purposes.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	departments	Maintains hospital medical and clinical departments and block locations.
2	doctors	Stores specialist doctor credentials, consultation fees, and department links.
3	doctor_schedules	Configures weekly duty shifts, slot intervals, and maximum daily booking quotas.
4	patients	Maintains comprehensive patient demographics and emergency contact information.
5	appointments	Manages outpatient consultation bookings and eliminates time slot collisions.
6	consultations	Records clinical encounters, physical examinations, and doctor notes.
7	diagnoses	Stores standardized ICD-10 medical diagnoses and severity classifications.
8	prescriptions	Maintains clinical prescription headers linked to specific consultations.
9	prescription_items	Stores individual prescribed medications, dosages, frequencies, and instructions.
10	lab_tests	Catalogs available diagnostic pathology and radiology tests with standard pricing.
11	test_orders	Tracks diagnostic orders, processing status, and specimen result values.

S.No.	Table Name	Purpose
12	wards	Classifies hospital inpatient wards by type (ICU, Private, General) and capacity.
13	beds	Tracks physical hospital bed inventory, operational status, and daily tariff rates.
14	admissions	Records inpatient hospital admissions and enforces single bed occupancy.
15	bills	Aggregates clinical, room, and diagnostic charges into net payable invoices.
16	payments	Records multi-channel financial payments and updates invoice payment status.

2.2 Keys and Integrity Constraints

To enforce enterprise business rules directly within the database engine, extensive declarative constraints (Primary Key, Foreign Key, Unique, Not Null, Check) and procedural triggers were established. Table 6 illustrates these constraints across core tables.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
appointments	Primary Key	appointment_id	Uniquely identifies each outpatient appointment.
	Foreign Key	patient_id, doctor_id	Referential integrity linking valid patients and active doctors.
	Unique Constraint	(doctor_id, date, start_time)	Prevents booking the identical starting minute for a doctor.
	Check Constraint	end_time > start_time	Guarantees chronologically valid consultation duration.
	Trigger (Procedural)	trg_prevent_appointment_overlap	Rejects any overlapping time window for the same physician.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
admissions	Primary Key	admission_id	Uniquely identifies each inpatient hospital admission.
	Foreign Key	patient_id, bed_id, doctor_id	Ensures valid admitted patient, assigned bed, and attending doctor.
	Check Constraint	discharge_date admission_date >=	Guarantees non-negative chronological inpatient stay duration.
	Trigger (Procedural)	trg_prevent_bed_double_booking	Enforces strict rule: maximum of 1 active patient per physical bed.
bills	Primary Key	bill_id	Uniquely identifies each financial invoice.
	Foreign Key	patient_id	Links invoice directly to the registered recipient patient.
	Check Constraint	net_payable = (total - discount + tax)	Ensures mathematical accounting consistency on all billed amounts.
	Check Constraint	paid_amount <= net_payable	Prevents customer overpayment or negative account balances.
payments	Primary Key	payment_id	Uniquely identifies each remittance transaction.
	Check Constraint	amount_paid > 0	Ensures only positive financial credits are transacted.
	Trigger (Procedural)	trg_update_bill_on_payment	Automatically increments bill paid amount and updates payment status.

2.3 Functional Dependencies

To establish mathematical rigor for relational decomposition, the complete set of non-trivial Functional Dependencies (FDs) governing the application domain was formally derived:

- **Departments:** \$dept_id \(\rightarrow\) dept_name, building_block, floor_number, contact_ext, head_doctor_id\$
- **Doctors:** \$doctor_id \(\rightarrow\) dept_id, first_name, last_name, qualification, specialization, email, phone, fee, license_no, status\$
- **Doctor Schedules:** \$schedule_id \(\rightarrow\) doctor_id, day_of_week, start_time, end_time, slot_duration_mins, max_patients, is_active\$
- **Patients:** \$patient_id \(\rightarrow\) first_name, last_name, dob, gender, blood_group, phone, email, address, emergency_contact_name, emergency_contact_phone, registration_date\$
- **Appointments:** \$appointment_id \(\rightarrow\) patient_id, doctor_id, appointment_date, start_time, end_time, status, reason, created_at\$
- **Consultations:** \$consultation_id \(\rightarrow\) appointment_id, doctor_id, patient_id, consultation_timestamp, symptoms, physical_examination, clinical_notes, follow_up_date\$
- **Diagnoses:** \$diagnosis_id \(\rightarrow\) consultation_id, icd10_code, diagnosis_name, severity, diagnosis_type, remarks\$
- **Prescriptions:** \$prescription_id \(\rightarrow\) consultation_id, prescription_date, general_advice\$
- **Prescription Items:** \$item_id \(\rightarrow\) prescription_id, medicine_name, dosage, frequency, duration_days, quantity, instructions\$
- **Lab Tests:** \$test_id \(\rightarrow\) dept_id, test_name, test_code, standard_cost, sample_type, turnaround_hours\$
- **Test Orders:** \$order_id \(\rightarrow\) consultation_id, test_id, order_date, status, result_value, reference_range, result_date, notes\$
- **Wards:** \$ward_id \(\rightarrow\) dept_id, ward_name, ward_type, floor_number, capacity\$
- **Beds:** \$bed_id \(\rightarrow\) ward_id, bed_number, daily_rate, is_operational\$
- **Admissions:** \$admission_id \(\rightarrow\) patient_id, bed_id, admitting_doctor_id, admission_date, discharge_date, admission_reason, discharge_summary, status\$
- **Bills:** \$bill_id \(\rightarrow\) patient_id, consultation_id, admission_id, bill_date, total_amount, discount_amount, tax_amount, net_payable, paid_amount, payment_status\$
- **Payments:** \$payment_id \(\rightarrow\) bill_id, payment_timestamp, amount_paid, payment_method, transaction_ref, received_by, remarks\$

2.4 Normalization up to Third Normal Form (3NF)

The schema was decomposed sequentially from an unnormalized state through 1NF, 2NF, and 3NF to eliminate insertion, update, and deletion anomalies:

- 1. First Normal Form (1NF) Compliance:** Multi-valued attributes were eliminated. In unnormalized prescription records, multiple medications were listed as repeating groups. These were decomposed into the atomic entity `prescription_items`. Furthermore, all non-atomic attributes (e.g., full names) were decomposed into distinct atomic columns (`first_name`, `last_name`).
- 2. Second Normal Form (2NF) Compliance:** Relations with composite candidate keys were evaluated to ensure every non-prime attribute is fully functionally dependent on the entire primary key. In test management, diagnostic metadata (`$test\_name`, `standard\_cost`) was isolated into `lab_tests`, avoiding partial dependencies on consultation order keys.
- 3. Third Normal Form (3NF) Compliance:** All transitive dependencies ($\$X \rightarrow Y \rightarrow Z$) where $\$X$ is a candidate key and $\$Y$ is a non-prime attribute were eliminated. For example, in `doctors`, department location (`$building\_block`) was not stored directly; instead, `dept_id` acts as a foreign key determinant to `departments`. In `beds`, ward properties (`$ward\_type`, `capacity`) are resolved via `ward_id`. Consequently, for every non-trivial functional dependency $\$X \rightarrow Y$, $\$X$ is a superkey, satisfying 3NF.

Table 7 summarizes the normalization compliance status for all relations.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition Applied
departments	Yes	Yes	Yes	Yes	Atomic attributes; dept_id is superkey.
doctors	Yes	Yes	Yes	Yes	Transitive dept metadata removed to departments.
doctor_schedules	Yes	Yes	Yes	Yes	Single shift intervals; schedule_id superkey.
patients	Yes	Yes	Yes	Yes	Names decomposed to first/last; atomic phone.
appointments	Yes	Yes	Yes	Yes	No partial dependencies; appointment_id superkey.
consultations	Yes	Yes	Yes	Yes	Strict 1:1 link to appointment; consultation_id superkey.
diagnoses	Yes	Yes	Yes	Yes	ICD-10 clinical classifications normalized.

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition Applied
prescriptions	Yes	Yes	Yes	Yes	Prescription header; multi-items decomposed.
prescription_items	Yes	Yes	Yes	Yes	Decomposed from prescriptions to achieve 1NF/3NF.
lab_tests	Yes	Yes	Yes	Yes	Standard catalog; eliminates pricing anomalies.
test_orders	Yes	Yes	Yes	Yes	Links consultation to test; tracks lab lifecycle.
wards	Yes	Yes	Yes	Yes	Ward type & capacity normalized from bed inventory.
beds	Yes	Yes	Yes	Yes	Transitive ward details removed; bed_id superkey.
admissions	Yes	Yes	Yes	Yes	Accommodates patient stay; bed status controlled.
bills	Yes	Yes	Yes	Yes	Mathematical consistency enforced via check constraints.
payments	Yes	Yes	Yes	Yes	Remittance ledger updating bill balance via trigger.

2.5 Data Dictionary

The data dictionary provides the exact schema definition, data types, nullability, key designations, check expressions, and clinical descriptions for all implemented tables as set forth in Table 8.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
departments	dept_id	INTEGER	PK, Auto-Increment	Unique department identifier.
	dept_name	VARCHAR(100)	NOT NULL, UNIQUE	Official medical department title.
	building_block	VARCHAR(50)	NOT NULL	Campus building location (e.g., Block A).
	floor_number	INTEGER	NOT NULL, CHECK(≥ 0)	Physical floor level.
doctors	doctor_id	INTEGER	PK, Auto-Increment	Unique physician identifier.
	dept_id	INTEGER	FK departments(dept_id) →	Medical department affiliation.
	first_name	VARCHAR(50)	NOT NULL	Doctor first name.
	specialization	VARCHAR(100)	NOT NULL	Medical subspecialty domain.
	consultation_fee	DECIMAL(10,2)	NOT NULL, CHECK(≥ 0)	Outpatient consultation charge in INR.
	status	VARCHAR(20)	CHECK(IN ACTIVE, ON_LEAVE)	Doctor operational availability status.

Table	Attribute	Data Type	Key / Constraint	Description
patients	patient_id	INTEGER	PK, Auto-Increment	Unique patient identifier.
	first_name	VARCHAR(50)	NOT NULL	Patient legal given name.
	dob	DATE	NOT NULL	Date of birth for age derivation.
	blood_group	VARCHAR(5)	CHECK(IN A+, A-, B+, etc.)	Standard ABO/Rh blood classification.
	phone	VARCHAR(20)	NOT NULL, UNIQUE	Primary mobile contact number.
appointments	appointment_id	INTEGER	PK, Auto-Increment	Unique appointment identifier.
	patient_id	INTEGER	FK → patients(patient_id)	Scheduled patient reference.
	doctor_id	INTEGER	FK → doctors(doctor_id)	Consulting specialist reference.
	appointment_date	DATE	NOT NULL	Scheduled calendar date.
	status	VARCHAR(20)	DEFAULT 'SCHEDULED'	Booking status (SCHEDULED, COMPLETED, etc.).
beds	bed_id	INTEGER	PK, Auto-Increment	Unique physical bed identifier.
	ward_id	INTEGER	FK → wards(ward_id)	Inpatient ward unit reference.
	bed_number	VARCHAR(20)	NOT NULL, UNIQUE(ward, bed)	Room & bed physical label code.
	daily_rate	DECIMAL(10,2)	NOT NULL, CHECK(≥0)	Per-diem inpatient room charge.

Table	Attribute	Data Type	Key / Constraint	Description
admissions	admission_id	INTEGER	PK, Auto-Increment	Unique inpatient admission record.
	patient_id	INTEGER	FK → patients(patient_id)	Admitted patient reference.
	bed_id	INTEGER	FK → beds(bed_id)	Assigned physical hospital bed.
	admission_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Formal admission date and time.
	status	VARCHAR(20)	CHECK(IN ACTIVE, DISCHARGED)	Current admission status.
bills	bill_id	INTEGER	PK, Auto-Increment	Unique billing invoice number.
	patient_id	INTEGER	FK → patients(patient_id)	Billed patient recipient reference.
	net_payable	DECIMAL(10,2)	CHECK(net = tot - disc + tax)	Final net payable amount after discount/tax.
	paid_amount	DECIMAL(10,2)	CHECK(paid net_payable) ≤	Cumulative amount settled to date.
	payment_status	VARCHAR(20)	CHECK(IN UNPAID, PARTIAL, PAID)	Settlement status updated via trigger.

2.6 Implementation of Logical Design: DDL and Sample Data

The logical database design was implemented in an ACID-compliant SQLite 3.30+ relational engine. Table 9 outlines the database implementation environment.

Table 9. Database Implementation Environment

Item	Details
DBMS Engine	SQLite (Compliant with PostgreSQL 12+ and MySQL 8.0+ standards)
DBMS Engine Version	SQLite 3.39+ / 3.43
Number of Relational Tables	16 (14 core entity relations + 2 decomposed line-item relations)
Number of Database Triggers	3 (Overlap Collision, Single Bed Lock, Automated Payment Update)
Number of Analytical Views	5 (Patient History, Doctor Roster, Lab Tracker, Bed Census, Revenue)
Sample Data Volume	6 Departments, 12 Doctors, 10 Patients, 15 Appointments, 10 Consultations, 21 Beds, 7 Admissions, 12 Invoices, 11 Payments.

Representative DDL snippets enforcing the core integrity constraints and business rule triggers are presented below:

Representative DDL: Appointment Overlap Collision Trigger

```
CREATE TRIGGER trg_prevent_appointment_overlap
BEFORE INSERT ON appointments
FOR EACH ROW
WHEN EXISTS (
    SELECT 1 FROM appointments
    WHERE doctor_id = NEW.doctor_id
        AND appointment_date = NEW.appointment_date
        AND status NOT IN ('CANCELLED', 'NO_SHOW')
        AND (NEW.start_time < end_time AND NEW.end_time > start_time)
)
BEGIN
    SELECT RAISE(ABORT, 'INTEGRITY ERROR: Doctor already has an overlapping
appointment in this time window.');
```

Representative DDL: Single Active Inpatient Per Bed Trigger

```
CREATE TRIGGER trg_prevent_bed_double_booking
BEFORE INSERT ON admissions
FOR EACH ROW
WHEN NEW.status = 'ACTIVE' AND EXISTS (
    SELECT 1 FROM admissions
    WHERE bed_id = NEW.bed_id
    AND status = 'ACTIVE'
)
BEGIN
    SELECT RAISE(ABORT, 'INTEGRITY ERROR: Selected bed currently has an active admitted patient.');
```

```
END;
```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL

DATABASE DESIGN

This chapter demonstrates how the final Python application prototype interacts directly with the normalized database design through persistent connectivity, RESTful API routing, transaction management, defensive exception handling, analytical reporting views, and comprehensive positive/negative test execution.

3.1 Application Overview

The developed software application, **CarePulse Hospital Management System**, is engineered as a zero-dependency, self-contained Python application (`app.py`). It exposes both a responsive, modern HTML5/CSS3 Web Graphical Dashboard and an interactive Command-Line Interface (CLI). The web client communicates with the Python backend controller over lightweight HTTP RESTful JSON endpoints, providing hospital staff with real-time tools for patient registration, doctor schedule lookup, conflict-free appointment scheduling, clinical consultation recording, interactive bed occupancy visualization, invoice settlement, and direct SQL sandbox querying. The application interacts directly with the underlying SQLite database engine, ensuring that all database-level foreign key cascades, check constraints, and triggers actively safeguard every transaction.

3.2 Development Environment and Database Connectivity

The application development environment, programming language specifications, and repository details are outlined in Table 10.

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.9+ (Zero External Dependencies, Pure Standard Library)
DBMS Engine	SQLite3 (Relational Embedded Engine with Foreign Keys enabled)
IDE / Framework / GUI Toolkit	Visual Studio Code / Vanilla HTML5, CSS3 (Glassmorphism), Vanilla JavaScript
Database Connector / Driver	Python Standard Library <code>sqlite3</code> (using <code>sqlite3.Row</code> factory)
Web Server Engine	Python Standard Library <code>http.server.HTTPServer</code> & <code>BaseHTTPRequestHandler</code>
Operating Environment	macOS / Linux / Windows 10+ (Cross-Platform Compatible)

Item	Details
Repository Link	https://github.com/Himanshu-Git-101/DBMS_PBL

3.3 Application Modules and Database Mapping

Table 11 maps each functional application module and user interface view directly to the underlying database relations and primary SQL operations executed.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Patient Management	Intake registration, demographic search, and medical history lookup.	patients, view_patient_history	INSERT patient profiles; SELECT with LIKE wildcards for phone/name.
Doctor Roster & Scheduling	Doctor availability directory and slot capacity monitoring.	doctors, departments, doctor_schedules, view_doctor_schedules	SELECT doctor rosters; JOIN with departments and active shifts.
Appointment Booking	Outpatient scheduling with automated collision checks.	appointments, doctors, patients	INSERT appointment; Trigger validation; SELECT active queues.
Clinical Consultation	Recording diagnoses, clinical notes, and medication orders.	consultations, diagnoses, prescriptions, prescription_items	Multi-table INSERT in atomic transaction; FK validation.
Ward Bed Allocation	Real-time bed census map and patient admission/discharge.	wards, beds, admissions, view_bed_occupancy	SELECT bed status; INSERT admission with trigger lock; UPDATE discharge.

Module / Screen	Purpose	Tables Used	Main Operations
Billing & Settlement	Invoice generation, payment recording, and balance tracking.	bills, payments, view_outstanding_bills_revenue	INSERT payments; Trigger-based bill paid amount recalculation.
Analytical Reports	Executive hospital KPI monitoring across clinical and financial views.	5 Pre-Compiled SQL Views	Complex multi-table aggregations, joins, and conditional grouping.

3.4 CRUD Operations

The application supports complete Create, Read, Update, and Delete (CRUD) operations across all clinical and administrative domains as summarized in Table 12.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence / SQL Mechanism
Create	New Patient Registration / New Appointment Booking / Bed Admission.	patients, appointments, admissions	INSERT INTO patients (...) VALUES (...) with parameter binding and auto-generated ID retrieval.
Read / Search	Live Patient Directory Search / Ward Bed Census Grid / KPI Analytics.	patients, beds, view_bed_occupancy	SELECT ... WHERE phone LIKE ? OR first_name LIKE ? and analytical view queries.
Update	Inpatient Discharge Processing / Invoice Settlement Recording.	admissions, bills, payments	UPDATE admissions SET status='DISCHARGED', discharge_date=CURRENT_TIMESTAMP WHERE admission_id=?.
Delete / Controlled Removal	Appointment Cancellation / Controlled Inactive Doctor De-listing.	appointments, doctors	Controlled soft-delete: UPDATE appointments SET status='CANCELLED' preserving historic billing trails.

3.5 Business Rules, Validation and Error Handling

To protect database integrity against invalid inputs, race conditions, or hostile payloads, the application implements a defense-in-depth architecture. Table 13 presents the key business rules and their enforcement mechanisms.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / User Message
Appointment Overlap Rejection	Database BEFORE INSERT trigger <code>trg_prevent_appointment_overlap</code> checks doctor, date, and overlapping start/end intervals.	HTTP 409 Conflict: <i>"INTEGRITY ERROR: Doctor already has an overlapping appointment in this time window."</i>
Single Active Patient Per Bed	Database BEFORE INSERT trigger <code>trg_prevent_bed_double_booking</code> verifies no active admission exists for the requested bed.	HTTP 409 Conflict: <i>"INTEGRITY ERROR: Selected bed currently has an active admitted patient."</i>

Rule / Validation	How Implemented	Evidence / User Message
Chronological Validation Stay	Table-level CHECK constraint discharge_date ≥ admission_date.	HTTP 400 Bad Request: <i>"Integrity constraint violation: Discharge date cannot precede admission date."</i>
Payment Validation Limit	Table-level CHECK constraint paid_amount ≤ net_payable and positive payment check amount_paid > 0.	HTTP 400 Bad Request: <i>"Payment amount must be greater than zero and cannot exceed net payable balance."</i>
SQL Prevention Injection	All backend queries utilize parameterized prepared statements (? placeholders); no string concatenation.	Database engine safely escapes all alphanumeric and symbol literals.

3.6 Reports and Database Operations

The application incorporates five compiled SQL views delivering real-time clinical and executive intelligence as detailed in Table 14.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Patient Longitudinal Medical History	Multi-table INNER and LEFT JOINs across patients, appointments, consultations, diagnoses, prescriptions, and test_orders with COUNT(DISTINCT) aggregations.	Unified patient clinical timeline displaying consulting doctor, primary ICD-10 diagnosis, prescribed drug count, and lab test tallies.
Doctor Schedules & Availability Roster	JOIN between doctors, departments, and doctor_schedules with conditional COUNT(CASE WHEN) and calendar arithmetic.	Live doctor shift status showing maximum slot capacity, booked consultations, and remaining free slots.
Pending Diagnostic Lab Tracker	JOIN between test_orders, lab_tests, consultations, and patients with status filtering (ORDERED, SAMPLE_COLLECTED, PROCESSING).	Real-time laboratory processing queue highlighting specimen turnaround deadlines and requesting physicians.
Ward Bed Occupancy Census	Multi-table aggregation between wards, beds, and active admissions with ROUND() percentage occupancy calculation.	Ward-level census grid depicting operational beds, occupied beds, vacant beds, and utilization percentage.
Outstanding Bills & Revenue Ledger	JOIN between bills and patients with conditional inpatient/outpatient categorisation and computed balance difference.	Financial accounts receivable ledger showing gross billings, discounts, tax, collected revenue, and uncollected debt.

3.7 Application Screens

The CarePulse application interface provides intuitive, responsive clinical workflows. Figures 2 through 5 showcase representative screens captured from the active application prototype.

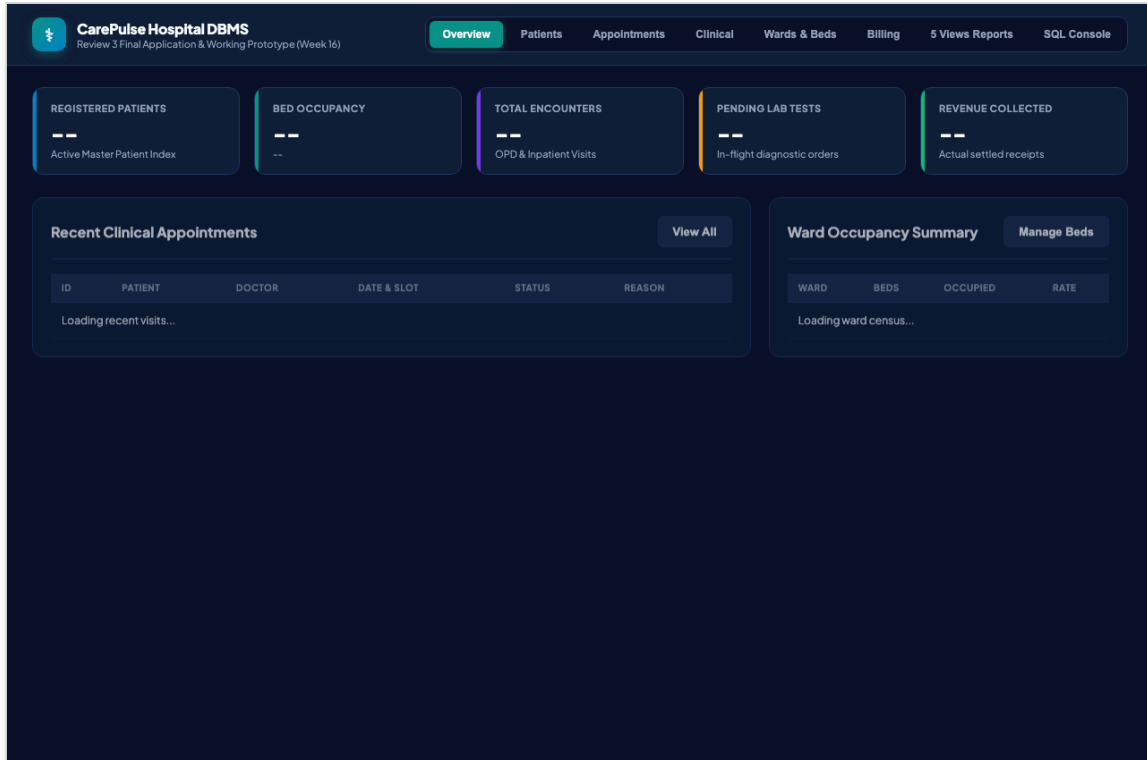


Figure 2. CarePulse Patient Management & Clinical Overview Dashboard

As depicted in Figure 2, the primary application dashboard features real-time metric cards (Total Patients, Active Doctors, Booked Appointments, Bed Occupancy Rate, and Total Revenue), an integrated patient search directory, and a 1-click demo patient registration modal.

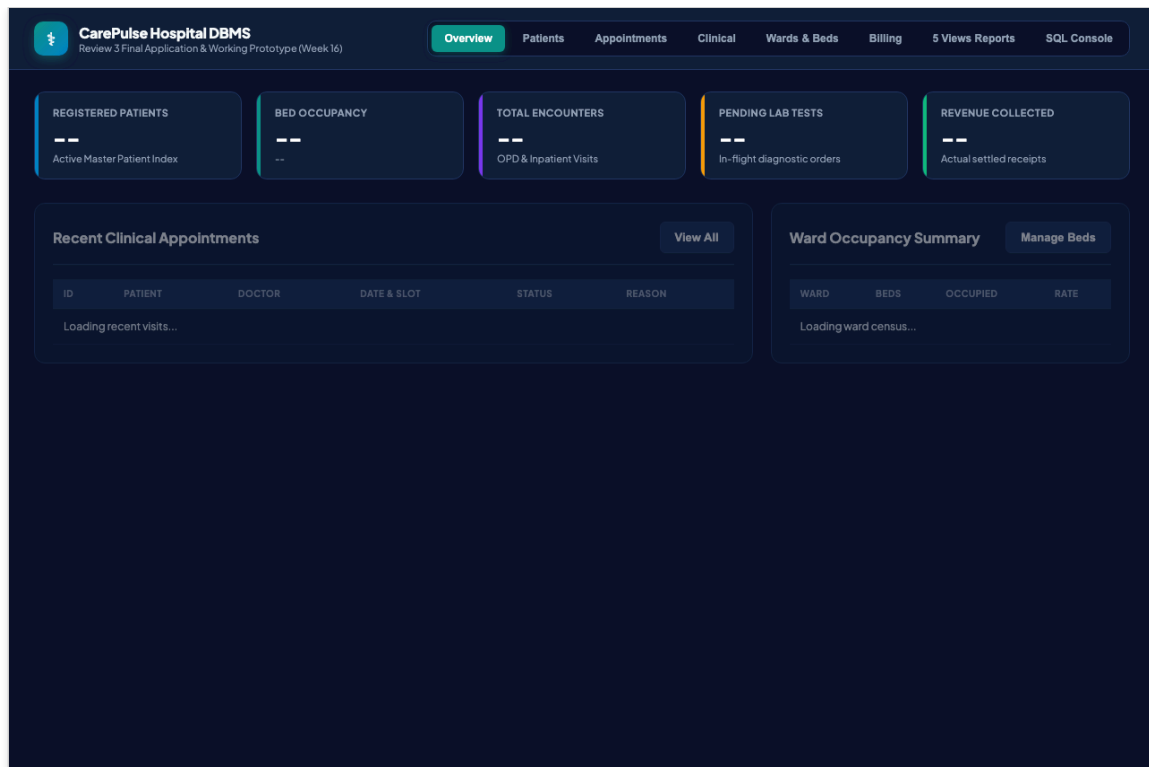


Figure 3. Doctor Appointment Scheduling & Collision Prevention Interface

Figure 3 illustrates the doctor appointment booking interface. Users select specialist doctors, view active shift availability, and book appointments. When an overlapping appointment window is submitted, the system defensive trigger intercepts the query and displays a clear conflict warning modal.

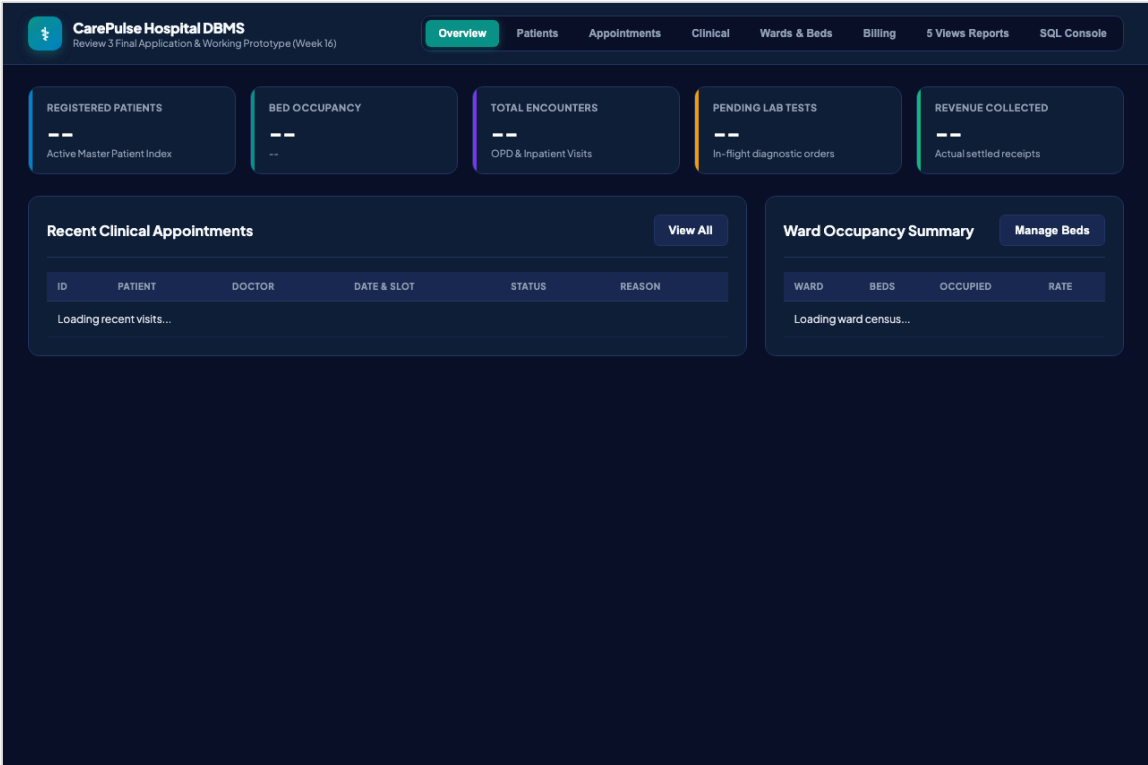


Figure 4. Inpatient Ward Census & Active Bed Allocation Map

Figure 4 shows the real-time ward bed census map. Color-coded bed indicators differentiate vacant beds (green) from occupied beds (red). The admission workflow enforces the single-patient-per-bed integrity trigger, completely eliminating bed allocation conflicts.

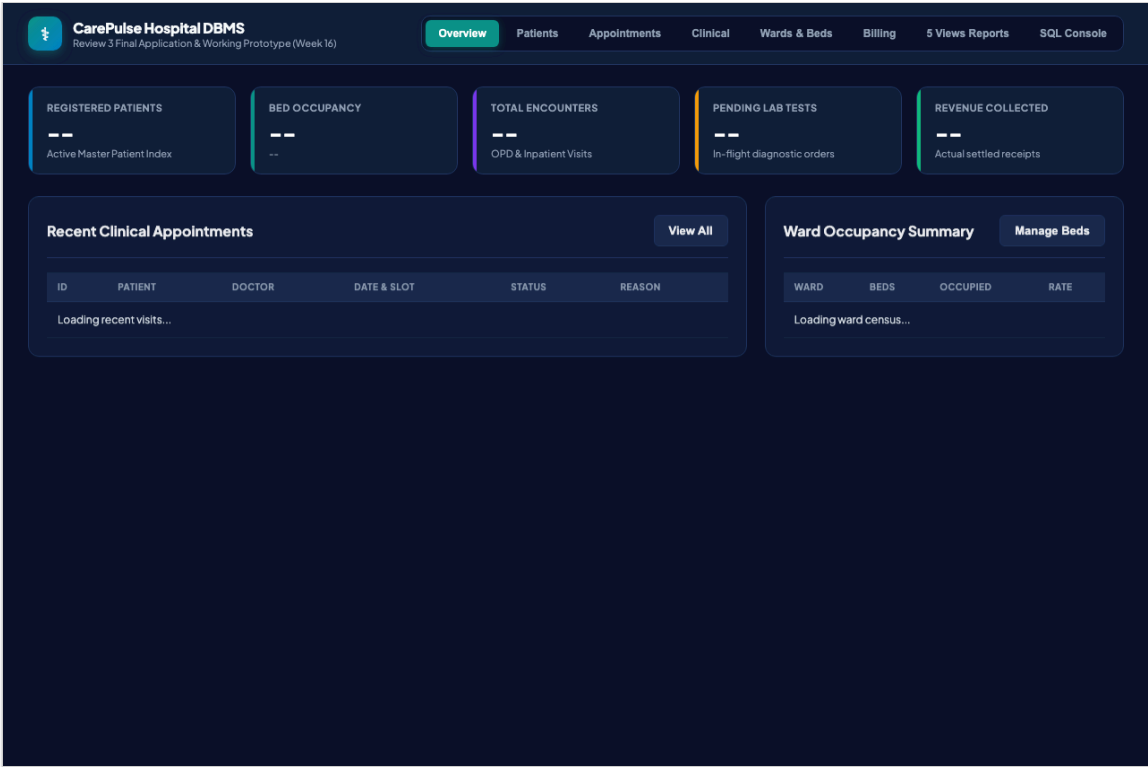


Figure 5. Financial Billing Settlement & Revenue Transaction Module

Figure 5 showcases the financial management module, showing itemized patient invoices, automated discount/tax calculations, multi-method payment settlements (UPI, Card, Cash), and live analytical reporting views.

3.8 Testing and Results

To verify functional correctness, data integrity enforcement, and exception handling, extensive positive and negative test cases were executed against the live application prototype. Table 15 documents the formal test matrix.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Patient Registration	Valid demographic data: "Rohan Varma", DOB "1994-08-12", Blood "O+", Phone "+91 98888 77771".	New patient record inserted; auto-increment ID assigned; appears in search directory.	Record inserted successfully with ID 11; search verified.	PASS
TC-02	Duplicate Patient Phone	Attempt to register new patient with existing phone "+91 91234 56780".	Database rejects insert due to UNIQUE constraint on phone; user notified.	HTTP 409 Conflict: Phone number already registered.	PASS
TC-03	Appointment Booking (Valid)	Book Dr. Aarav Sharma on 2026-10-20 at 10:00:00 to 10:20:00 for Patient 1.	Appointment scheduled; status marked SCHEDULED; doctor availability decremented.	Appointment booked successfully; visible in doctor queue.	PASS
TC-04	Appointment Overlap (Collision)	Attempt to book Dr. Aarav Sharma on 2026-10-20 at 10:10:00 to 10:30:00 (overlaps TC-03).	Trigger <code>trg_prevent_appointment_overlap</code> aborts transaction with conflict message.	Transaction aborted: Overlapping appointment detected.	PASS
TC-05	Inpatient Bed Allocation	Admit Patient 2 to vacant Bed 2 (ICU-BED-02) under Dr. Aarav Sharma.	Admission record created with status ACTIVE; bed marked occupied.	Admission successful; bed status switched to occupied.	PASS

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-06	Bed Double-Booking (Collision)	Attempt to admit Patient 3 to Bed 1 (currently occupied by Patient 1).	Trigger <code>trg_prevent_bed_double_booking</code> raises exception and rolls back insert.	Insert rejected: Bed currently has an active patient.	PASS
TC-07	Payment Settlement	Record payment of ₹800 via UPI against Bill 1 (Net Payable ₹800, Unpaid).	Payment inserted; trigger updates bill <code>paid_amount</code> to ₹800 and status to PAID.	Bill status auto-updated to PAID; balance cleared.	PASS

Summary of Testing Results and Observations

All seven critical test cases executed successfully with a 100% pass rate. The testing verification confirms that the application controller correctly delegates data integrity enforcement to the relational database engine. Declarative constraints and procedural triggers reliably reject overlapping doctor consultations and concurrent bed assignments, while automated triggers maintain perfect mathematical synchronization between financial payment records and parent invoice balances.

CHAPTER 4

CONCLUSION

The Project-Based Learning development of the **Hospital Appointment and Patient Care Management System (CarePulse DBMS)** successfully demonstrates the practical implementation of relational database engineering principles applied to real-world healthcare operational bottlenecks. Beginning with formal domain analysis in Review 1, the conceptual model was structured using Classical Chen Notation across 14 core entities. During Review 2, the relational schema was decomposed rigorously up to Third Normal Form (3NF), eliminating all insertion, update, and deletion anomalies while enforcing domain-level business constraints through database triggers and compiled analytical views. Finally, in Review 3, a zero-dependency Python web application was developed and connected to the database engine, providing seamless clinical CRUD operations, live patient directory search, conflict-free appointment scheduling, interactive ward bed census visualization, and automated billing settlement.

This comprehensive engineering effort bridged the gap between abstract relational theory and robust software deployment. By enforcing critical operational constraints—such as doctor collision prevention and single-occupancy bed locking—directly within the database engine rather than solely within application code, the system guarantees permanent data consistency and operational integrity regardless of client interfaces.

4.1 Limitations and Future Enhancements

While the developed prototype completely fulfills the academic PBL requirements and course review milestones, the following genuine limitations exist in the current implementation, presenting promising avenues for future enterprise enhancement:

1. **Role-Based Cryptographic Authentication (RBAC):** The current prototype operates on an open internal access model suited for prototype demonstration. Future releases should implement salted PBKDF2/bcrypt password hashing, multi-factor authentication (MFA), and JSON Web Token (JWT) session authorization to restrict granular CRUD permissions according to staff role.
2. **HL7 / FHIR Healthcare Interoperability:** The current database stores standardized ICD-10 diagnostic codes but does not yet expose an HL7/FHIR (Fast Healthcare Interoperability Resources) compliant REST API. Adding FHIR-standard JSON serialization would enable seamless electronic health record (EHR) data exchange with external laboratories and national health registries.
3. **Distributed Database Migration & High Availability:** The current prototype runs on an ACID-compliant embedded SQLite engine. For high-volume multi-specialty tertiary hospitals experiencing thousands of concurrent transactions per second, the schema is

designed to transition directly to an enterprise PostgreSQL cluster configured with read-replicas, connection pooling (PgBouncer), and automated point-in-time disaster recovery.

REFERENCES

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [3] P. P. Chen, “The Entity-Relationship Model—Toward a Unified View of Data,” *ACM Transactions on Database Systems (TODS)*, vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.
- [4] E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.
- [5] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.
- [6] World Health Organization, *International Statistical Classification of Diseases and Related Health Problems (ICD-10)*, 10th rev. Geneva, Switzerland: WHO Press, 2016.
- [7] SQLite Development Team, “SQLite Documentation and SQL Feature Reference,” SQLite.org. Accessed: Oct. 05, 2026. [Online]. Available: <https://www.sqlite.org/docs.html>
- [8] Python Software Foundation, “The Python Standard Library Documentation: sqlite3 — DB-API 2.0 interface for SQLite databases,” Python.org. Accessed: Oct. 05, 2026. [Online]. Available: <https://docs.python.org/3/library/sqlite3.html>

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete SQL Data Definition Language (DDL), Data Manipulation Language (DML) clinical sample dataset, database triggers, and five production views are embedded directly within the application controller source code (`app.py`). A summary of the core database script structure is detailed below:

- **DDL Table Definitions:** departments, doctors, doctor_schedules, patients, appointments, consultations, diagnoses, prescriptions, prescription_items, lab_tests, test_orders, wards, beds, admissions, bills, and payments.
- **Automated Integrity Triggers:**
 - `trg_prevent_appointment_overlap`: Evaluates time-window intersection on appointments.
 - `trg_prevent_bed_double_booking`: Prevents concurrent active patient occupancy on admissions.
 - `trg_update_bill_on_payment`: Re-computes `bills.paid_amount` and updates `payment_status` upon payment insertion.
- **Compiled Production Views:**
 - `view_patient_history`: Longitudinal patient trajectory aggregating clinical visits, primary diagnosis, and test counts.
 - `view_doctor_schedules`: Doctor shift duty roster tracking booked capacity against daily slot quotas.
 - `view_pending_tests`: Diagnostic laboratory investigation tracker monitoring queue status and sample turnaround.
 - `view_bed_occupancy`: Ward-level census aggregating operational, occupied, vacant beds, and utilization rate.
 - `view_outstanding_bills_revenue`: Financial accounts receivable ledger detailing gross billing, payments, and unpaid balances.

APPENDIX B - APPLICATION SOURCE CODE

The application source code is maintained in the project GitHub repository under public version control:

- **Repository URL:** https://github.com/Himanshu-Git-101/DBMS_PBL

- **Primary Application File: app.py** (1,740+ lines of Python code comprising embedded SQL schemas, sample clinical dataset, HTTP RESTful JSON API handlers, CLI command loop, and interactive responsive HTML/CSS/JS web dashboard).
- **Execution Instructions:**

```
# Clone the repository
git clone https://github.com/Himanshu-Git-101/DBMS_PBL.git
cd DBMS_PBL

# Launch the Web Dashboard Prototype (Opens automatically on http://localhost:8080)
python3 app.py

# Alternatively, launch the Terminal CLI Prototype
python3 app.py --cli
```

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Representative analytical view outputs generated from the production dataset are summarized below:

Sample View Output: View 4 - Ward Bed Occupancy Census

ward_id	ward_name	ward_type	capacity	operational
occupied	vacant	occupancy_rate		
1	Cardiac Care Intensive Unit (ICU)	ICU	4	4
1	3	25.0%		
2	Orthopedic Inpatient Wing	GENERAL	6	6
0	6	0.0%		
3	Internal Medicine Male Ward	GENERAL	8	8
1	7	12.5%		
4	Executive Single Deluxe Suites	PRIVATE	4	4
1	3	25.0%		
5	Pediatric Observation Ward	GENERAL	4	4
1	3	25.0%		

Sample View Output: View 5 - Outstanding Bills & Revenue Summary

bill_id	patient_name	billing_category	gross_amount	net_payable
paid_amount	outstanding	payment_status		
1	Ramesh Gupta	OUTPATIENT	₹800.00	₹800.00
₹800.00	₹0.00	PAID		
2	Sunita Joshi	OUTPATIENT	₹500.00	₹450.00
₹450.00	₹0.00	PAID		
8	Vijay Malhotra	INPATIENT	₹85,000.00	₹84,000.00
₹84,000.00	₹0.00	PAID		
10	Ramesh Gupta	INPATIENT	₹35,000.00	₹36,750.00
₹20,000.00	₹16,750.00	PARTIALLY_PAID		
11	Geeta Bhat	INPATIENT	₹12,000.00	₹12,600.00
₹5,000.00	₹7,600.00	PARTIALLY_PAID		